

Error Analysis

Carrying Out Error Analysis

What is Error Analysis

If we are trying to get a learning algorithm to do a task that humans can do, and our algorithm is not yet at the performance of a human.

→ Then manually examining mistakes that your algorithm is making, can give you insights into what to do next

This process is called **error analysis**.

Here's a **clean summary of the key points**:

- **Don't guess where to improve—measure it.**

Use **error analysis** before spending months optimizing a specific problem.

Look at dev examples to evaluate ideas



Should you try to make your cat classifier do better on dogs? ←

Error analysis: → 5-10 min

- Get ~100 mislabeled dev set examples.
- Count up how many are dogs.

Handwritten calculations:

5% → 5/100

10% → 9.5%

50% → 50/100

10% → 5%

Handwritten notes: "ceiling", "90% accuracy", "10% error"

not to spend the next few months focused on trying to find solutions to

Andrew Ng

- **Procedure:**

1. Take ~100 misclassified examples from the dev set.
2. **Manually inspect** them.
3. **Categorize errors** (e.g., dogs, great cats, blurry images).
4. **Count percentages** for each category.

- **Ceiling analysis (upper bound on improvement):**

- If only **5%** of errors are dogs → fixing dogs can at best reduce error from **10%** → **9.5%** (small gain).
- If **50%** of errors are dogs → fixing dogs could reduce error from **10%** → **5%** (big gain).
- The fraction of errors tells you **how worthwhile** an idea is.

Evaluate multiple ideas in parallel

Ideas for cat detection:

- Fix pictures of dogs being recognized as cats ←
- Fix great cats (lions, panthers, etc..) being misrecognized ←
- Improve performance on blurry images ←

Image	Dog	Great Cats	Blurry	Instagram	Comments
1	✓			✓	Pitbull
2			✓	✓	
3		✓	✓		Rainy day at zoo
⋮	⋮	⋮	⋮		
% of total	8%	43%	61%	12%	

Andrew Ng

- **Compare multiple ideas in parallel:**

Use a table/spreadsheet with columns for different error types (dogs, great cats, blurry images, etc.).

- **Categories can overlap** (one image can be both "blurry" and "great cat").

- **New error categories can emerge** during inspection (e.g., Instagram filters) — add them dynamically.

- **Outcome:**

Error analysis doesn't force a decision, but it **guides prioritization**, showing which improvements have the **highest potential impact**.

- **Big benefit:**

A 5–10 minute manual review can save months of wasted work.

Cleaning Up Incorrectly Labeled Data

Incorrectly labeled examples



DL algorithms are quite robust to random errors in the training set.

enough and the actual percentage of errors is maybe not too high

Andrew Ng

Key points summary:

- **Clarify terms:**

- *Mislabeled examples*: model predicts the wrong label.
- *Incorrectly labeled examples*: the dataset's ground-truth label (Y) is wrong due to human error.

- **Training set label errors:**

- Deep learning models are generally **robust to random label noise**.
- If incorrect labels are **random and relatively few**, it's often **not worth spending much time fixing them**.

- Fixing them can help, but is usually optional if the dataset is large.
- **Important caveat – systematic errors:**
 - Models are **not robust to systematic labeling mistakes**.
 - Example: if labelers consistently mark white dogs as cats, the model will learn this wrong pattern.
 - Systematic bias in labels **should be fixed**.
- **Bottom line for training data:**
 - Random errors → usually okay to ignore.
 - Systematic errors → dangerous, worth correcting.
- **Open question raised:**
 - What to do about **incorrect labels in the dev set or test set** (to be discussed next).

Error analysis

Image	Dog	Great Cat	Blurry	Incorrectly labeled	Comments
...					
98				✓	Labeler missed cat in background
99		✓			
100				✓	Drawing of a cat; Not a real cat.
% of total	8%	43%	61%	6%	

Overall dev set error	10%	
Errors due incorrect labels	0.6%	←
Errors due to other causes	9.4%	←

2%	✓
0.6%	←
1.4%	

Goal of dev set is to help you select between two classifiers A & B.

Andrew Ng

Here's the **key-point summary** of this section:

- **Track incorrect labels during error analysis**
 - Add an extra column to your error-analysis table for **incorrect ground-truth labels (wrong Y)**.
 - Count how many "errors" are actually caused by **labeling mistakes**, not model mistakes.
- **Deciding whether to fix dev/test labels depends on impact**
 - Fix incorrect labels **only if they significantly affect your ability to evaluate models**.
 - Otherwise, it may not be worth the time.
- **Three numbers to look at**
 1. **Overall dev set error** (e.g., 10%)
 2. **Fraction of errors due to incorrect labels** (e.g., 6%)
 3. **Errors due to other causes** (remaining error)
- **Case 1: High overall error**
 - Example: 10% total error, 6% of errors due to incorrect labels
 - Label noise contributes only **0.6% absolute error**
 - Most error (9.4%) comes from real model issues
→ **Not urgent** to fix dev labels
- **Case 2: Low overall error**
 - Example: 2% total error, same 0.6% due to incorrect labels
 - Now **30% of dev errors** come from bad labels

- Remaining model error is only 1.4%

→ **Worth fixing dev labels**

- **Why this matters**

- The dev set's job is to **compare models reliably**
- If label errors are large enough to obscure differences (e.g., 2.1% vs 1.9%), you **can't trust dev set conclusions**

- **Bottom line**

- When model performance is still poor → fixing dev labels is low priority
- When model performance is strong → dev label errors become a **major bottleneck** and should be fixed

Correcting incorrect dev/test set examples

- Apply same process to your dev and test sets to make sure they continue to come from the same distribution
- Consider examining examples your algorithm got right as well as ones it got wrong.
- Train and dev/test data may now come from slightly different distributions.

I will talk more about how to handle this later this week. Andrew Ng

Here's a **concise key-point summary** of this final part on fixing labels and error analysis:

- **Fix dev & test sets together**

- Apply the **same label-fixing process to both dev and test sets**.
- This keeps them from the **same distribution**, which is critical for reliable evaluation and generalization.

- **Don't only inspect errors**

- Ideally, also re-check **examples the model got right**, not just wrong ones.
- Fixing only wrong predictions can **bias evaluation** and give the model an unfair advantage.
- In practice, this is often skipped because checking "correct" cases can be very time-consuming (e.g., 98% of data).

- **Training set label fixes are optional**

- It's usually **less important** to fix labels in the training set.
- Deep learning models tolerate label noise in training data fairly well.
- It's acceptable to fix **only dev/test labels**, even if training data remains noisier.

- **Distribution mismatch is acceptable (to a degree)**

- Dev and test sets must match each other closely.
- Training data can come from a slightly different distribution and still work well.

- **Human insight still matters**

- Despite "just train on more data," **manual error analysis is extremely valuable** in applied ML.
- A short amount of time (minutes to hours) reviewing examples can **dramatically improve prioritization**.

- **Practical advice**

- Don't avoid manually inspecting data—it's not glamorous, but it's highly effective.
- Personally reviewing errors helps you understand failure modes and choose the **best next steps**.

👉 Core takeaway:

Careful, consistent error analysis—especially on dev/test sets—is one of the highest-ROI activities in practical machine learning.

Speech recognition example



- • Noisy background
 - • Café noise
 - • Car noise
- • Accented speech
- • Far from microphone
- • Young children's speech
- • Stuttering *uh, ah, um, ...*
- • ...

- • Set up dev/test set and metric
- Build initial system quickly
- Use Bias/Variance analysis & Error analysis to prioritize next steps.

Andrew Ng

to recap, what I recommend you do is build your first system quickly, then iterate.

Speech recognition example



- • Noisy background
 - • Café noise
 - • Car noise
- • Accented speech
- • Far from microphone
- • Young children's speech
- • Stuttering
- • ...

- • Set up dev/test set and metric
- Build initial system quickly
- Use Bias/Variance analysis & Error analysis to prioritize next steps.

Andrew Ng

And if you're trying to build a face recognizer,

Guideline:

Build your first system quickly, then iterate

Here's a **clear, structured summary of the key ideas**:

- **Core advice: build fast, then iterate**
 - For a *new* ML application, don't overthink the first system.
 - Build an initial version **quickly**, even if it's "quick and dirty".
- **Why this matters**
 - Most ML problems have **many possible improvement directions** (e.g., noise, accents, far-field speech, children's speech).
 - It's hard to know what to prioritize **before seeing real errors**.
- **Recommended initial steps**
 1. **Define dev/test sets and an evaluation metric** (set a target).
 2. **Train a basic initial model** and measure its performance.
- **Use analysis to guide improvements**
 - Apply **bias/variance analysis** to see if you're underfitting or overfitting.
 - Apply **error analysis** to identify the dominant sources of mistakes.

- Let the data tell you **what to fix next** (e.g., far-field speech errors).
- **Value of the first system**
 - Not accuracy, but **information**:
 - Reveals failure modes
 - Shows which directions matter most
 - Prevents wasted effort on low-impact ideas
- **When this advice applies less**
 - If you have **strong prior experience** in the domain.
 - If there's a **large, directly applicable research literature** (e.g., face recognition).
- **Common mistake**
 - Teams often **overcomplicate** the first system.
 - A simpler early system is usually more effective for learning what to do next.
- **Bottom line**
 - If your goal is to **build a working system (not invent new algorithms)**:
 - Build something simple and fast
 - Analyze its errors
 - Iterate based on evidence, not guesses